HiDRA: A Blazing Fast LM-Head Replacement

Satyajit Kumar, Henry Ndubuaku
Cactus Compute
{satyajit, henry}@cactuscompute.com

Abstract

The output projection (LMHead) in large language models scales linearly with vocabulary size and constitutes a major portion of decoding cost, often at least 10% on sub-1B models which are often deployed on small consumer hardware. We propose HiDRA (Hierarchical Decision Routing Architecture), a post-training method that replaces the dense vocabulary projection with a hierarchical sequence of binary decisions, reducing the computational complexity from $\mathcal{O}(V)$ to $\mathcal{O}(\log V)$ dot products per token. The hierarchy is constructed on top of an existing model by hierarchically splitting the embedding space in half by a single hyperplane. We prove that minimizing the error rate of a given hyperplane is equivalent to a Linear Discriminant Analysis optimization problem. On Gemma with WildChat completions, HiDRA achieves an $8.04\times$ practical CPU speedup for the full output stage including routing, while reducing top-1 accuracy by 4.2% absolute at a moderate routing depth with beam search. More generally, different depths provide a smooth spectrum of operating points, allowing practitioners to select the desired balance between compute and model quality without retraining the base model.

1 Introduction

In modern language models, the final projection layer (LMHead) computes logits

$$z_k = h^\top e_k,$$

where $h \in \mathbb{R}^d$ is the hidden state and $\{e_k\}_{k=1}^V$ are token embeddings. This operation requires a dense matrix multiplication of cost $O(Vd)$ per decoding step, which scales linearly with vocabulary size. This projection often dominates inference latency on edge devices, where CPU-based execution is common, and large LMHead matrix multiplications can significantly impede sustained generation.

At the same time, the probability mass of the output distribution is typically concentrated on a small fraction of the vocabulary. We aim to reduce the number of projections the model has to perform, decreasing output-projection latency and alleviating sustained CPU load during decoding.

We propose HiDRA (Hierarchical Decision Routing Architecture), a replacement for the LMHead that uses a binary decision tree over tokens. Each token is assigned a binary path of length $L \approx \log_2 V$. At inference time, the hidden state is routed through a sequence of hyperplane decisions requiring only $O(Ld)$ computation, followed by evaluating LMHead just on the candidate tokens at the leaf. This reduces the effective computation from $O(Vd)$ to $O(Ld + (V/2^L)d)$.

We provide a theoretical analysis showing that this problem is aligned with the Fisher Linear Discriminant, an existing optimization problem. This motivates the initialization for the decision vectors that we use to construct the hyperplanes. This leads to stable routing and improved empirical performance. A key property of the method is that the tree can be truncated at any depth, yielding a continuous tradeoff between speed and accuracy without retraining or architectural changes.

2 Related Work

Large-vocabulary neural language models have long relied on hierarchical output layers to reduce the cost of the softmax projection. Early work introduced hierarchical softmax for neural language

modeling, replacing the flat vocabulary distribution with a tree-structured factorization to achieve logarithmic complexity in vocabulary size (Morin & Bengio, 2005; Mnih & Hinton, 2008). This approach was later popularized in large-scale word embedding models (Mikolov et al., 2013) and extended to neural machine translation and other large-vocabulary settings (Jean et al., 2015). Mi et al. (Mi et al., 2016) introduce vocabulary shortlist pruning for neural machine translation, where a small, input-dependent target vocabulary is constructed using lexical and alignment cues, allowing decoding to avoid scoring the full vocabulary while largely preserving translation quality. More recent variants, such as adaptive softmax, further exploit hierarchical structure to improve training efficiency on modern hardware (Grave et al., 2017). However, these methods have primarily targeted training efficiency or embedding learning, and have not been widely adopted to accelerate the decode-time output projection in modern autoregressive large language models. In light of this hierarchical embedding research, we must understand the structure of the embedding space of modern LLMs. Prior work has shown that these spaces tend to be highly *anisotropic*, meaning that variance is concentrated in a few dominant directions rather than being evenly distributed. Mu et al. (2017) showed that removing the top principal components of word embeddings improves semantic performance, indicating that these directions encode shared, non-discriminative information. Ethayarajh (2019) further demonstrated that contextual embeddings from models such as BERT and GPT-2 occupy narrow cones, with similarity dominated by common components rather than meaningful structure. More recent work argues that such anisotropy is a fundamental property of neural representations learned during training, rather than a simple artifact of optimization (Rudman & Eickhoff, 2024). These findings suggest that token embeddings do not form clean clusters, making centroid-based grouping methods such as k-means less suitable. Linear Discriminant Analysis (LDA) finds linear projections that maximize class separation while minimizing within-class variance (Fisher, 1936). Its discriminative objective remains useful in modern deep embedding learning (Wu et al., 2016; Bartan & Pilanci, 2022) and pruning/efficient classification (Tian et al., 2017), suggesting that Fisher-style projections are well-suited for constructing efficient decision boundaries in high-dimensional neural embedding spaces.

3 METHODS

We design HiDRA as a lightweight modification to the LMHead of an existing language model. Instead of scoring the full vocabulary with a dense projection, we construct a hierarchy of binary decision vectors that progressively partition the token set. These decision vectors are initialized using Fisher LDA projections derived from the token embeddings, and are then refined using supervised training. At inference time, routing can terminate at any depth under beam routing, after which the original LMHead scores only the tokens in the reached partitions.

LDA-based decision vectors Let $e_k \in \mathbb{R}^d$ be the LMHead embedding for token k and $h \in \mathbb{R}^d$ the hidden state. Standard decoding computes $\arg\max_k h^\top e_k$ at cost $O(Vd)$. HiDRA arranges tokens in a binary tree; each internal node ℓ predicts $\mathrm{bit}_\ell = \mathbf{1}[h^\top d_\ell + b_\ell > 0]$. Since h is unknown at construction time, we model $h = e_t + \rho$ with $\rho \sim \mathcal{N}(0, \Sigma)$, where t is the true next token. Define the margin $m_t := d^\top e_t + b$. An error occurs when $\mathrm{sign}(d^\top h + b) \neq \mathrm{sign}(m_t)$, i.e. $d^\top h + b = m_t + d^\top \rho$ flips sign. Because $d^\top \rho \sim \mathcal{N}(0, d^\top \Sigma d)$, a Gaussian tail bound yields

$$P(\text{error}) \leq 2 \exp\left(-\frac{m_t^2}{2\, d^\top \Sigma d}\right), \qquad \text{so we maximize } \frac{m_t^2}{d^\top \Sigma d}.$$

Optimal direction. For a node splitting tokens into L vs. R with means μ_L, μ_R and within-class covariance Σ_w, the expected margin scales with $d^\top(\mu_R - \mu_L)$. Since the true noise covariance is unknown, we approximate it using the within-class covariance Σ_w, which empirically aligns well with the structure of LLM embedding spaces. Thus we solve the Fisher objective

$$\max_d \frac{(d^\top(\mu_R - \mu_L))^2}{d^\top \Sigma_w d},$$

whose maximizer satisfies $d^\star \propto \Sigma_w^{-1}(\mu_R - \mu_L)$. Hence, under anisotropic noise, the optimal decision direction is the LDA separator; we use these LDA vectors to initialize internal nodes.

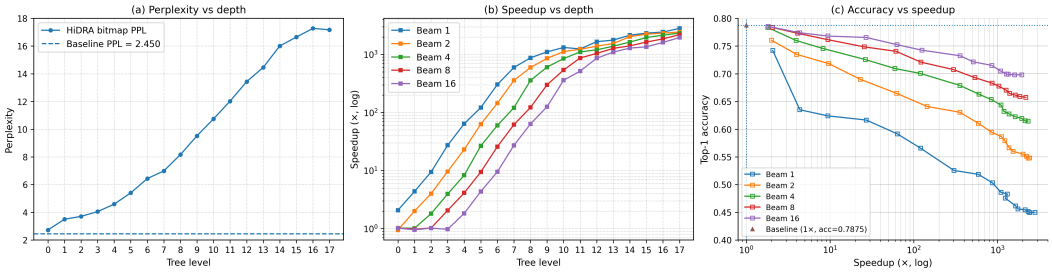


Figure 1: HiDRA evaluation on Gemma 3 270M-IT (WildChat). (a) Perplexity increases with tree depth. (b) Projection-only speedup, excluding routing overhead, grows exponentially with depth and decreases with beam width. (c) Beam routing improves top-1 accuracy at fixed speedups.

Decision vector training While LDA provides a principled initialization, it ignores token frequency and the true distribution of hidden states. We therefore fine-tune each decision vector using teacher-forced hidden states, training a binary classifier at each node with a binary cross-entropy loss. This adapts routing boundaries to real model activations observed on WildChat. For each node ℓ, we train a binary classifier to predict the correct routing bit:

$$\mathcal{L}_\ell = \text{BCE}(\hat{b}_\ell, b_\ell^{\text{true}}).$$

Training is performed on model completions from the WildChat dataset, allowing the decision boundaries to adapt to the true distribution of hidden states and token frequencies.

Hierarchical probability and early stopping Each internal node ℓ outputs a routing probability $p_\ell = \sigma(h^\top d_\ell + b_\ell)$. For a token k with path bits $(b_1, \ldots, b_L)$, the induced hierarchical probability factorizes as

$$p(k) = \prod_{\ell=1}^{L} p_\ell^{b_\ell} (1 - p_\ell)^{1 - b_\ell},$$

which defines a valid distribution over the vocabulary via the tree structure. At inference time, routing may terminate early at depth $l < L$. In this case, the model evaluates the original LMHead only over the tokens in the reached subtree, yielding a truncated distribution. This enables a continuous speed–accuracy tradeoff by controlling the routing depth and beam width. Because early stopping does not normalize over the full vocabulary, the resulting distribution differs from the base model softmax. Consequently, perplexity values are not directly comparable to baseline perplexity and should be interpreted only as a heuristic diagnostic of routing quality.

Beam routing Greedy routing restricts the candidate tokens to a single branch of the tree, which may exclude high-probability tokens that lie across nearby partitions. To address this, we use beam routing. At each level, the top k routing paths are retained based on their cumulative probabilities. The final candidate set is formed from the union of the tokens in these beam-selected partitions. This allows the model to consider tokens from multiple regions of the embedding space, improving accuracy while still providing substantial computational savings.

4 Experiments

4.1 Setup

We evaluate HiDRA on **Gemma 3 270M-IT** (Team et al., 2025) using **WildChat** prompt–completion data (Zhao et al., 2024). The base language model is kept frozen; only the routing tree is trained. We construct a depth-18 hierarchical bitmap tree over the Gemma vocabulary ($|V| = 262{,}144$) using a Fisher LDA–motivated median-split procedure on the input embedding matrix. This produces per-token path bitmaps and per-node split parameters. We then train the tree's decision vectors with next-token supervision on WildChat. Training uses sequence length 512, batch size 16, for 5 epochs with AdamW (learning rate 3×10^{-4}). We use a 90k/5k train/validation split,

Technique	Depth l	Perplexity	Top-1 Acc	Projection-only ms/token	Route-only ms/token
HiDRA Beam 1	2	**3.71**	0.624	1.16	0.0302
HiDRA Beam 2	3	4.05	0.7185	1.14	0.0310
HiDRA Beam 4	4	4.59	0.7455	1.32	0.0327
HiDRA Beam 8	5	5.40	0.7615	1.16	0.0351
HiDRA Beam 16	6	6.43	**0.768**	1.15	0.0481
Baseline LMHead	–	2.45	0.7875	10.95	–

Table 1: Snapshot of HiDRA operating points on Gemma 3 270M-IT (WildChat). The "Projection-only" column isolates the restricted LMHead computation after routing. The "Route-only" column reports fused native CPU routing latency for the same depth/beam setting from the separate 8-thread route-only benchmark. Baseline LMHead has no routing stage.

evaluating every 1000 steps. At inference, hidden states are routed to depth l, after which the original LM head scores only tokens in the reached subtree. We report next-token top-1 accuracy relative to the base model, perplexity computed from the hierarchical routing distribution, and projection-only speed measured as milliseconds per token for the restricted LMHead computation after routing. These timings exclude routing overhead and the base model forward pass. Projection speed is evaluated on CPU across depths 0–17 and beam widths $k \in \{1, 2, 4, 8, 16\}$ using 100 warmup and 1000 timed iterations per setting.

4.2 Results

Figure 1 summarizes the projection-speed–accuracy tradeoff of HiDRA across routing depths and beam widths. Table 1 highlights representative operating points. At the representative operating point $(l = 4, k = 4)$ with 4 CPU threads, HiDRA reduces isolated output-stage latency from 15.33 ± 0.04 to 1.91 ± 0.02 ms/token ($8.04 \pm 0.09\times$), which translates into a teacher-forced full-step decode reduction from **61.08 to 49.11 ms/token** ($1.24\times$) because transformer forward time remains dominant. We intentionally isolate the output projection in Figure 1(b) and Table 1 to expose the scaling behavior of the LMHead itself; however, fused native CPU routing remains small in practice, staying below 0.1 ms/token across all depths and beam widths. Figure 1(a) shows perplexity increasing monotonically with depth, largely independent of beam width, indicating that perplexity is primarily controlled by the truncation depth rather than multi-path routing. Figure 1(b) shows that projection-only speedup grows approximately exponentially with depth, while larger beam widths reduce speed by increasing the number of candidate tokens that must be scored. Figure 1(c) demonstrates that beam routing substantially recovers top-1 accuracy at fixed speedups. Additional analyses appear in the appendix. Appendix A.1 summarizes teacher-forced decode across CPU thread counts. Appendix A.3 shows that a WildChat-trained router retains a similar fraction of dense-model top-1 accuracy on held-out WildChat and out-of-domain FineWeb-EDU.

5 Conclusion

We introduced HiDRA, a hierarchical decision routing layer that replaces dense full-vocabulary LMHead scoring with a sequence of binary routing decisions followed by restricted candidate scoring. On Gemma, HiDRA provides substantial speedups while maintaining strong top-1 accuracy, and beam routing further improves accuracy at fixed operating points. Because routing depth is chosen at inference time, HiDRA offers a continuous spectrum of speed–quality tradeoffs that can be adapted to hardware or latency constraints. HiDRA is mathematically grounded in an LDA formulation and operates as a lightweight post-training modification that does not retrain the base model. With a fused CPU router, routing overhead remains small relative to dense LMHead scoring, yielding an $8.04\times$ speedup for the full output stage including routing at a representative operating point; in teacher-forced autoregressive decoding, this translates into a practical $1.24\times$ end-to-end speedup because transformer forward time remains dominant. Evaluations on held-out FineWeb-EDU further indicate that a WildChat-trained router preserves a similar fraction of base-model accuracy out of domain even when absolute task difficulty changes substantially. Overall, HiDRA shows that the LMHead can be replaced with a hierarchical routing structure that yields real system-level gains without modifying or retraining the underlying model.

REFERENCES

Burak Bartan and Mert Pilanci. Neural Fisher discriminant analysis: Optimal neural network embeddings in polynomial time. In Kamalika Chaudhuri, Stefanie Jegelka, Le Song, Csaba Szepesvari, Gang Niu, and Sivan Sabato (eds.), *Proceedings of the 39th International Conference on Machine Learning*, volume 162 of *Proceedings of Machine Learning Research*, pp. 1647–1663. PMLR, 17–23 Jul 2022. URL https://proceedings.mlr.press/v162/bartan22a.html.

Kawin Ethayarajh. How contextual are contextualized word representations? comparing the geometry of bert, elmo, and gpt-2 embeddings, 2019. URL https://arxiv.org/abs/1909.00512.

Rory A. Fisher. The use of multiple measurements in taxonomic problems. *Annals of Human Genetics*, 7:179–188, 1936. URL https://api.semanticscholar.org/CorpusID:29084021.

Edouard Grave, Armand Joulin, Moustapha Cissé, David Grangier, and Hervé Jégou. Efficient softmax approximation for gpus, 2017. URL https://arxiv.org/abs/1609.04309.

Sébastien Jean, Kyunghyun Cho, Roland Memisevic, and Yoshua Bengio. On using very large target vocabulary for neural machine translation, 2015. URL https://arxiv.org/abs/1412.2007.

Haitao Mi, Zhiguo Wang, and Abe Ittycheriah. Vocabulary manipulation for neural machine translation, 2016. URL https://arxiv.org/abs/1605.03209.

Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. Efficient estimation of word representations in vector space, 2013. URL https://arxiv.org/abs/1301.3781.

Andriy Mnih and Geoffrey E Hinton. A scalable hierarchical distributed language model. In D. Koller, D. Schuurmans, Y. Bengio, and L. Bottou (eds.), *Advances in Neural Information Processing Systems*, volume 21. Curran Associates, Inc., 2008. URL https://proceedings.neurips.cc/paper_files/paper/2008/file/1e056d2b0ebd5c878c550da6ac5d3724-Paper.pdf.

Frederic Morin and Yoshua Bengio. Hierarchical probabilistic neural network language model. In Robert G. Cowell and Zoubin Ghahramani (eds.), *Proceedings of the Tenth International Workshop on Artificial Intelligence and Statistics*, volume R5 of *Proceedings of Machine Learning Research*, pp. 246–252. PMLR, 06–08 Jan 2005. URL https://proceedings.mlr.press/r5/morin05a.html. Reissued by PMLR on 30 March 2021.

Jiaqi Mu, Suma Bhat, and Pramod Viswanath. All-but-the-top: Simple and effective postprocessing for word representations. *CoRR*, abs/1702.01417, 2017. URL http://arxiv.org/abs/1702.01417.

William Rudman and Carsten Eickhoff. Stable anisotropic regularization, 2024. URL https://arxiv.org/abs/2305.19358.

Gemma Team, Aishwarya Kamath, Johan Ferret, Shreya Pathak, Nino Vieillard, Ramona Merhej, Sarah Perrin, Tatiana Matejovicova, Alexandre Ramé, Morgane Rivière, Louis Rouillard, Thomas Mesnard, Geoffrey Cideron, Jean bastien Grill, Sabela Ramos, Edouard Yvinec, Michelle Casbon, Etienne Pot, Ivo Penchev, Gaël Liu, Francesco Visin, Kathleen Kenealy, Lucas Beyer, Xiaohai Zhai, Anton Tsitsulin, Robert Busa-Fekete, Alex Feng, Noveen Sachdeva, Benjamin Coleman, Yi Gao, Basil Mustafa, Iain Barr, Emilio Parisotto, David Tian, Matan Eyal, Colin Cherry, Jan-Thorsten Peter, Danila Sinopalnikov, Surya Bhupatiraju, Rishabh Agarwal, Mehran Kazemi, Dan Malkin, Ravin Kumar, David Vilar, Idan Brusilovsky, Jiaming Luo, Andreas Steiner, Abe Friesen, Abhanshu Sharma, Abheesht Sharma, Adi Mayrav Gilady, Adrian Goedeckemeyer, Alaa Saade, Alex Feng, Alexander Kolesnikov, Alexei Bendebury, Alvin Abdagic, Amit Vadi, András György, André Susano Pinto, Anil Das, Ankur Bapna, Antoine Miech, Antoine Yang, Antonia Paterson, Ashish Shenoy, Ayan Chakrabarti, Bilal Piot, Bo Wu, Bobak Shahriari, Bryce Petrini, Charlie Chen, Charline Le Lan, Christopher A. Choquette-Choo, CJ Carey, Cormac Brick, Daniel

Deutsch, Danielle Eisenbud, Dee Cattle, Derek Cheng, Dimitris Paparas, Divyashree Shivakumar Sreepathihalli, Doug Reid, Dustin Tran, Dustin Zelle, Eric Noland, Erwin Huizenga, Eugene Kharitonov, Frederick Liu, Gagik Amirkhanyan, Glenn Cameron, Hadi Hashemi, Hanna Klimczak-Plucińska, Harman Singh, Harsh Mehta, Harshal Tushar Lehri, Hussein Hazimeh, Ian Ballantyne, Idan Szpektor, Ivan Nardini, Jean Pouget-Abadie, Jetha Chan, Joe Stanton, John Wieting, Jonathan Lai, Jordi Orbay, Joseph Fernandez, Josh Newlan, Ju yeong Ji, Jyotinder Singh, Kat Black, Kathy Yu, Kevin Hui, Kiran Vodrahalli, Klaus Greff, Linhai Qiu, Marcella Valentine, Marina Coelho, Marvin Ritter, Matt Hoffman, Matthew Watson, Mayank Chaturvedi, Michael Moynihan, Min Ma, Nabila Babar, Natasha Noy, Nathan Byrd, Nick Roy, Nikola Momchev, Nilay Chauhan, Noveen Sachdeva, Oskar Bunyan, Pankil Botarda, Paul Caron, Paul Kishan Rubenstein, Phil Culliton, Philipp Schmid, Pier Giuseppe Sessa, Pingmei Xu, Piotr Stanczyk, Pouya Tafti, Rakesh Shivanna, Renjie Wu, Renke Pan, Reza Rokni, Rob Willoughby, Rohith Vallu, Ryan Mullins, Sammy Jerome, Sara Smoot, Sertan Girgin, Shariq Iqbal, Shashir Reddy, Shruti Sheth, Siim Põder, Sijal Bhatnagar, Sindhu Raghuram Panyam, Sivan Eiger, Susan Zhang, Tianqi Liu, Trevor Yacovone, Tyler Liechty, Uday Kalra, Utku Evci, Vedant Misra, Vincent Roseberry, Vlad Feinberg, Vlad Kolesnikov, Woohyun Han, Woosuk Kwon, Xi Chen, Yinlam Chow, Yuvein Zhu, Zichuan Wei, Zoltan Egyed, Victor Cotruta, Minh Giang, Phoebe Kirk, Anand Rao, Kat Black, Nabila Babar, Jessica Lo, Erica Moreira, Luiz Gustavo Martins, Omar Sanseviero, Lucas Gonzalez, Zach Gleicher, Tris Warkentin, Vahab Mirrokni, Evan Senter, Eli Collins, Joelle Barral, Zoubin Ghahramani, Raia Hadsell, Yossi Matias, D. Sculley, Slav Petrov, Noah Fiedel, Noam Shazeer, Oriol Vinyals, Jeff Dean, Demis Hassabis, Koray Kavukcuoglu, Clement Farabet, Elena Buchatskaya, Jean-Baptiste Alayrac, Rohan Anil, Dmitry, Lepikhin, Sebastian Borgeaud, Olivier Bachem, Armand Joulin, Alek Andreev, Cassidy Hardin, Robert Dadashi, and Léonard Hussenot. Gemma 3 technical report, 2025. URL https://arxiv.org/abs/2503.19786.

Qing Tian, Tal Arbel, and James J. Clark. Efficient gender classification using a deep lda-pruned net. *CoRR*, abs/1704.06305, 2017. URL http://arxiv.org/abs/1704.06305.

Lin Wu, Chunhua Shen, and Anton van den Hengel. Deep linear discriminant analysis on fisher networks: A hybrid architecture for person re-identification. *CoRR*, abs/1606.01595, 2016. URL http://arxiv.org/abs/1606.01595.

Wenting Zhao, Xiang Ren, Jack Hessel, Claire Cardie, Yejin Choi, and Yuntian Deng. Wildchat: 1m chatgpt interaction logs in the wild, 2024. URL https://arxiv.org/abs/2405.01470.

A Appendix

A.1 Teacher-forced Decode Across Thread Counts

Table 2 summarizes the teacher-forced autoregressive decode benchmark at the representative operating point $(l = 4, k = 4)$ for 4 and 8 CPU threads. These measurements should not be read as a direct replacement for the $8.04\times$ output-stage benchmark in the main text: they measure full autoregressive decoding with KV-cache reuse under explicit thread settings, whereas the main-text output-stage result isolates cached-hidden LMHead computation on a single token across repeated trials.

Threads	Forward only	Dense decode	HiDRA decode	Decode speedup
4	51.29	61.08	49.11	1.24×
8	52.01	63.11	50.23	1.26×

Table 2: Teacher-forced autoregressive decode benchmark at $(l = 4, k = 4)$. All columns report ms/token except "Decode speedup". "Forward only" measures the transformer step with KV-cache reuse and no LMHead scoring; "Dense decode" and "HiDRA decode" report end-to-end per-token latency for the full dense and HiDRA decoders, respectively. Because this benchmark measures a different scope than the direct single-token output-stage benchmark in the main text, the resulting speedups are not expected to match exactly.

l	B1			B2			B4			B8			B16		
	Acc	Proj	Route	Acc	Proj	Route	Acc	Proj	Route	Acc	Proj	Route	Acc	Proj	Route
0	0.7420	5.2669	0.0301	0.7875	11.5298	0.0297	0.7875	10.7012	0.0297	0.7875	10.7021	0.0298	0.7875	10.7240	0.0296
1	0.6350	2.5007	0.0296	0.7605	5.4488	0.0301	0.7875	10.7765	0.0300	0.7875	11.1853	0.0302	0.7875	11.4689	0.0297
2	0.6240	1.1572	0.0302	0.7350	2.7300	0.0306	0.7835	6.0444	0.0306	0.7875	10.6815	0.0306	0.7875	10.7403	0.0305
3	0.6165	0.4022	0.0301	0.7185	1.1357	0.0310	0.7600	2.7700	0.0314	0.7835	5.3083	0.0317	0.7875	11.2196	0.0322
4	0.5915	0.1711	0.0303	0.6900	0.4742	0.0313	0.7455	1.3183	0.0327	0.7730	2.6551	0.0332	0.7855	5.9799	0.0341
5	0.5655	0.0900	0.0305	0.6645	0.1736	0.0319	0.7255	0.4110	0.0332	0.7615	1.1561	0.0351	0.7740	2.5120	0.0375
6	0.5255	0.0361	0.0309	0.6410	0.0757	0.0320	0.7095	0.1832	0.0420	0.7485	0.4245	0.0445	0.7680	1.1524	0.0481
7	0.5185	0.0184	0.0309	0.6305	0.0307	0.0324	0.7005	0.0906	0.0346	0.7405	0.1775	0.0392	0.7655	0.4036	0.0438
8	0.5030	0.0125	0.0315	0.6105	0.0184	0.0326	0.6790	0.0307	0.0352	0.7210	0.0897	0.0406	0.7525	0.1723	0.0484
9	0.4860	0.0099	0.0319	0.5945	0.0128	0.0333	0.6630	0.0181	0.0365	0.7075	0.0365	0.0423	0.7425	0.0871	0.0513
10	0.4830	0.0083	0.0317	0.5865	0.0098	0.0335	0.6535	0.0129	0.0391	0.6930	0.0203	0.0501	0.7325	0.0305	0.0607
11	0.4755	0.0088	0.0316	0.5790	0.0089	0.0415	0.6435	0.0099	0.0451	0.6830	0.0126	0.0516	0.7215	0.0212	0.0640
12	0.4615	0.0066	0.0319	0.5665	0.0079	0.0422	0.6320	0.0091	0.0459	0.6775	0.0105	0.0534	0.7150	0.0127	0.0668
13	0.4560	0.0062	0.0319	0.5600	0.0071	0.0425	0.6275	0.0079	0.0464	0.6705	0.0086	0.0545	0.7050	0.0100	0.0696
14	0.4545	0.0051	0.0323	0.5550	0.0054	0.0429	0.6225	0.0067	0.0470	0.6645	0.0078	0.0557	0.6995	0.0085	0.0721
15	0.4515	0.0047	0.0324	0.5510	0.0049	0.0431	0.6195	0.0056	0.0477	0.6615	0.0067	0.0576	0.6990	0.0081	0.0761
16	0.4495	0.0045	0.0325	0.5475	0.0047	0.0434	0.6150	0.0051	0.0486	0.6590	0.0059	0.0584	0.6980	0.0068	0.0784
17	0.4500	0.0039	0.0328	0.5480	0.0045	0.0438	0.6145	0.0047	0.0493	0.6575	0.0050	0.0606	0.6980	0.0056	0.0807
Baseline	0.7875	10.9455	–	0.7875	10.9455	–	0.7875	10.9455	–	0.7875	10.9455	–	0.7875	10.9455	–

Table 3: Rounded WildChat top-1 accuracy and latency sweep for all beam widths. "Proj" reports projection-only latency and "Route" reports route-only latency; all latencies are in ms/token.

A.2 Full Routing and Projection Sweep

Table 3 lists rounded top-1 accuracies and latencies for the full WildChat sweep shown in Figure 1. "Proj" reports the restricted LMHead timing used in the main figure, while "Route" reports fused native CPU routing latency from the separate 8-thread route-only benchmark for the same depth/beam operating point.

A.3 Dataset Dependence Across Domains

Table 4 compares held-out WildChat and FineWeb-EDU under the same evaluation harness, beam width $k = 4$, and 10,240 scored positions per dataset. It reports dense-model top-1 accuracy and the fraction of that accuracy retained by HiDRA at representative depths, showing that normalized transfer remains similar even though absolute task difficulty differs across domains.

Domain	Dense top-1	$l = 4$	$l = 8$	$l = 12$	$l = 17$
WildChat holdout	74.0	95.8	87.1	80.3	78.8
FineWeb-EDU	31.3	95.8	89.8	82.9	79.5

Table 4: Dense-model top-1 accuracy (%) and retained base-model top-1 accuracy (%) for a WildChat-trained router evaluated on held-out WildChat and out-of-domain FineWeb-EDU. Columns $l \in \{4, 8, 12, 17\}$ report $(\text{HiDRA top-1})/(\text{dense LM top-1}) \times 100$.